

Aligning Multiple Knowledge Graphs in A Single Pass

Yaming Yang

School of Computer Science and
Technology, Xidian University
Xi'an, China
yym@xidian.edu.cn

Zhe Wang

School of Computer Science and
Technology, Xidian University
Xi'an, China
zwang_01@stu.xidian.edu.cn

Ziyu Guan*

School of Computer Science and
Technology, Xidian University
Xi'an, China
zyguan@xidian.edu.cn

Wei Zhao

School of Computer Science and
Technology, Xidian University
Xi'an, China
ywzhao@mail.xidian.edu.cn

Weigang Lu

The Hong Kong University of Science
and Technology
Hong Kong SAR, China
weiganglu@ust.hk

Xinyan Huang

School of Artificial Intelligence,
Xidian University
Xi'an, China
huangxy@xidian.edu.cn

Jiangtao Cui

School of Computer Science and
Technology, Xidian University
Xi'an, China
cuijt@xidian.edu.cn

Xiaofei He

State Key Laboratory of CAD&CG,
Zhejiang University
Hangzhou, China
xiaofeihe@cad.zju.edu.cn

Abstract

Entity alignment (EA) is to identify equivalent entities across different knowledge graphs (KGs), which can help fuse these KGs into a more comprehensive one. Previous EA methods mainly focus on aligning a pair of KGs, and to the best of our knowledge, no existing EA method considers aligning multiple (more than two) KGs. To fill this research gap, in this work, we study a novel problem of aligning multiple KGs and propose an effective framework named MultiEA to solve the problem. First, we embed the entities of all the candidate KGs into a common feature space by a shared KG encoder. Then, we explore three alignment strategies to minimize the distances among pre-aligned entities. In particular, we propose an innovative inference enhancement technique to improve the alignment performance by incorporating high-order similarities. Finally, to verify the effectiveness of MultiEA, we construct two new real-world benchmark datasets and conduct extensive experiments on them. The results show that our MultiEA can effectively and efficiently align multiple KGs in a single pass. We release the source code at: <https://github.com/wzCSDN/MultiEA>.

CCS Concepts

• **Computing methodologies** → **Knowledge representation and reasoning**; **Neural networks**; **Supervised learning**; • **Information systems** → **Information integration**; **Data mining**.

Keywords

Knowledge Graphs, Entity Alignment, Graph Neural Networks

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792111>

ACM Reference Format:

Yaming Yang, Zhe Wang, Ziyu Guan, Wei Zhao, Weigang Lu, Xinyan Huang, Jiangtao Cui, and Xiaofei He. 2026. Aligning Multiple Knowledge Graphs in A Single Pass. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792111>

1 Introduction

Knowledge graphs (KGs) are a special kind of graph that can store a wealth of structural facts (i.e. knowledge) about the real world. Each fact is usually structured as a triple (h, r, t) , representing that head (subject) entity h and tail (object) entity t hold relation r between them. In recent years, KGs have successfully supported many web applications such as search engines [52], question-answer systems [9, 22, 32, 38], recommender systems [36, 38, 39], etc.

In practice, different KGs are constructed based on diverse data sources and different extraction approaches, and a single KG can usually cover only a specific aspect of structural facts. For example, an English KG usually contains more facts about the English-speaking society, while a Chinese KG contains more facts about the Chinese-speaking society. Considering that more KGs together can provide more comprehensive structural facts from various aspects, researchers have proposed many methods (Cf. surveys [30, 49, 53]) to fuse a pair of KGs into a unified one. The general process is to first identify equivalent entities between the candidate KGs, and then let them serve as the “bridge entities” to link the candidate KGs into a unified one.

Technically, existing mainstream methods typically solve this problem by embedding entities into a common latent representation space and minimizing the distance between pre-aligned entity pairs. This process is also referred to as entity alignment (EA). Depending on how to obtain the entity embeddings, they can be classified into two main categories: (1) Trans-based EA methods [5, 8, 25, 27, 50, 54] learn entity embeddings by letting each triple satisfy some specific geometric properties in the embedding space. Most of these

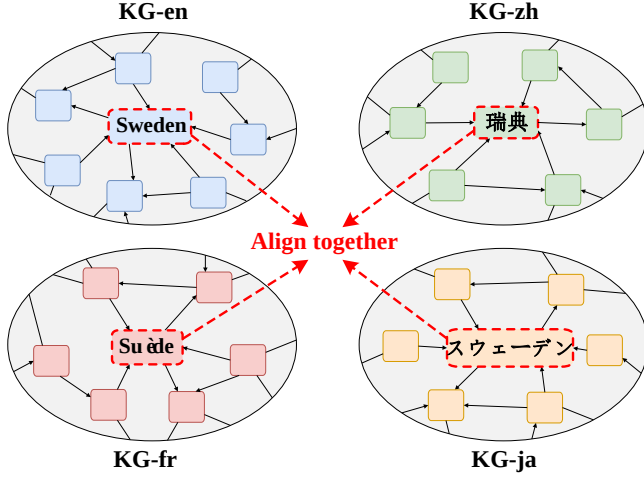


Figure 1: An example of aligning four KGs.

methods adopt TransE [3] as the translation module to preserve the property: $h + r \approx t$. (2) GNN-based EA methods [4, 11, 16, 18, 19, 21, 29, 40, 42–45, 47, 55] adopt graph neural network (GNN) models such as graph convolutional network (GCN) [10] to learn entity embeddings by iteratively aggregating the embeddings of neighbor entities.

However, all the existing EA methods are specifically designed for aligning only a pair of candidate KGs, which can cover limited knowledge for downstream applications. To the best of our knowledge, **none of the existing EA methods consider aligning multiple (more than two) KGs**. For example, Fig. 1 shows a case where four candidate KGs need to be aligned, which triggers a more challenging research problem.

There are three obstacles that prevent traditional pair-wise KG alignment methods from being applied to the multiple KG alignment problem studied in this work. First, existing EA datasets are particularly constructed for pair-wise KG alignment. At present, there is no benchmark dataset that can support the study of aligning multiple KGs. Second, if we trivially adapt existing methods to this task, they need to be separately executed multiple times since they can only utilize the local (pair-wise) alignment information in a single pass, which is inefficient. Last and most importantly, they cannot capture some useful global (beyond pair-wise) alignment information, affecting the final alignment performance. For the example in Fig. 1, when they align KG-en with KG-fr, and align KG-en with KG-zh independently, the entities of the three KGs will be projected into four separate embedding spaces. Consequently, it is easy to yield inconsistent results like $e_1^{(en)} \equiv e_1^{(fr)}$, $e_1^{(en)} \equiv e_1^{(zh)}$, and $e_1^{(fr)} \equiv e_2^{(zh)}$ (wrong), violating the transitivity of the alignment relationship.

To fill the research gap in this field, in this work, we innovatively study the problem of aligning multiple KGs in a single pass. First of all, to facilitate the study of this problem, we construct two new benchmark datasets that contain multiple KGs to be aligned. Then, we formulate a novel research problem of aligning multiple KGs concurrently. Finally, we propose an effective framework named

MultiEA to solve the problem. As illustrated in Fig. 1, our MultiEA can concurrently align the four candidate KGs in a single pass, which is more practical. In addition, it embeds the entities of all the candidate KGs into a unified space and thus can capture useful global alignment information, which helps achieve better alignment performance. Recall the example above, if MultiEA pulls $e_1^{(en)}$ and $e_1^{(fr)}$ closer and pulls $e_1^{(en)}$ and $e_1^{(zh)}$ closer. Then, $e_1^{(fr)}$ and $e_1^{(zh)}$ will definitely be pulled closer as well, leading to the consistent result: $e_1^{(en)} \equiv e_1^{(fr)}$, $e_1^{(en)} \equiv e_1^{(zh)}$, and $e_1^{(fr)} \equiv e_1^{(zh)}$ (correct).

The main contributions of this work are summarized as follows.

- To the best of our knowledge, in the EA research community, we are the first to study the problem of concurrently aligning multiple (more than two) KGs. We are also the first to construct the benchmark datasets and define the evaluation metric for this problem.
- We propose an effective framework named MultiEA to solve the problem. Three alignment strategies are explored to capture the global alignment information. An innovative inference enhancement technique is proposed to significantly boost the alignment performance.
- We conduct extensive experiments on the two constructed benchmark datasets. The results demonstrate that MultiEA can effectively and efficiently align multiple KGs in a single pass. We release our source code to facilitate further research on this problem.

2 Related Work

In this section, we comprehensively review existing methods from four aspects as follows.

Trans-based EA Methods. Early EA methods exploit KG embedding methods [1], e.g., TransE [3], to project a pair of candidate KGs into a common low-dimensional Euclidean space by letting the embeddings of head entity h , relation r , and tail entity t satisfy the triangular relationship: $h + r \approx t$. Given a set of pre-aligned entity pairs (i.e., seed alignment labels), they minimize the distance between these equivalent entity pairs in the embedding space. Thus, more potential equivalent entity pairs can be discovered after model optimization. Representative Trans-based EA methods include JEWP [8], JAPE [25], MTransE [5], IPTransE [54], BootEA [27], TransEdge [28], MultiKE [50], etc.

GNN-based EA Methods. Motivated by the extraordinary success of graph neural networks (GNNs) in extracting the structural features of graphs, GCN-Align [40] first introduces GCN [10] into EA, which has shown superior performance than Trans-based EA Methods, e.g., JEWP [8], MTransE [5], and JAPE [25]. Further, AVR-GCN [47] deftly combines the graph convolutional operator in GNNs and the translation operator in Trans-based methods to extract better entity embeddings. Afterward, a series of GNN-based EA methods, e.g., [4, 6, 11, 16, 18, 19, 21, 26, 29, 31, 33, 42–45, 55] are proposed. Recently, RREA [19] insightfully summarizes KG embedding-based EA methods and GNN-based EA methods into a unified framework consisting of two modules: shape-builder and alignment. The former constraints KGs into a specific distribution in

the embedding space, and the latter minimizes the distance between the embeddings of pre-aligned entity pairs.

3 Preliminaries

In this section, we first give the formal definition of KGs. Then, we formulate the novel problem of aligning multiple KGs.

Knowledge Graph (KG). A knowledge graph is defined as $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where $\mathcal{E}$ is the set of entities (nodes) and $\mathcal{R}$ is the set of relations (edges). $\mathcal{T} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ is the set of triples, and each triple $t \in \mathcal{T}$ is represented as $\langle e_i, r_k, e_j \rangle$, which means that head entity $e_i \in \mathcal{E}$ and tail entity $e_j \in \mathcal{E}$ hold relation $r_k \in \mathcal{R}$ between them. We use $\mathbf{h}_i \in \mathbb{R}^d$ to denote the representation of entity e_i in one model layer, where d is the dimensionality of embeddings. We use $\mathbf{g}_k \in \mathbb{R}^d$ to denote the embedding of relation r_k , which is a randomly initialized learnable parameter vector.

Multiple KG Alignment. The input is a set of KGs, denoted as $\{\mathcal{G}^{(1)}, \mathcal{G}^{(2)}, \mathcal{G}^{(m)}, \dots, \mathcal{G}^{(M)}\}$, where M is the number of the KGs, and $M > 2$. The m -th KG is denoted by $\mathcal{G}^{(m)} = (\mathcal{E}^{(m)}, \mathcal{R}^{(m)}, \mathcal{T}^{(m)})$. The problem is to identify equivalent entities that refer to the same real-world thing across all the M KGs, based on a set of seed alignment labels (pre-aligned entities), denoted as $\mathcal{S} = \{l_1, l_2, \dots, l_n, \dots, l_N\}$, where N is the number of labels. The n -th label is denoted as $l_n = (e_n^{(1)}, e_n^{(2)}, \dots, e_n^{(m)}, \dots, e_n^{(M)})$, where entity $e_n^{(m)} \in \mathcal{E}^{(m)}$, and all the M entities associated with l_n are equivalent.

As shown in Fig. 1, KG-en, KG-zh, KG-fr, and KG-ja are four candidate KGs that need to be aligned, where the four entities marked with red dashed lines refer to the same object, i.e., “Sweden”. We treat the four equivalent entities as one of the seed alignment labels, to help our EA algorithm find more potential equivalent entities. Finally, we can link (merge) the four KGs based on these detected equivalent entities across the four KGs.

4 Framework

In this section, we describe our proposed MultiEA framework in detail. Firstly, we design a GNN-like KG encoder to embed the entities of all the candidate KGs into a common vector space. Then, we propose three alignment strategies to measure the distances among pre-aligned entities, which guide the training of EA models. Finally, we develop an innovative inference enhancement technique to boost the alignment performance by incorporating high-order similarities.

4.1 KG Encoding

Given a set of KGs: $\{\mathcal{G}^{(1)}, \mathcal{G}^{(2)}, \dots, \mathcal{G}^{(m)}, \dots, \mathcal{G}^{(M)}\}$, we use a shared KG-oriented GNN encoder to compute their entity embeddings. In the following, we take the m -th KG, i.e., $\mathcal{G}^{(m)} = (\mathcal{E}^{(m)}, \mathcal{R}^{(m)}, \mathcal{T}^{(m)})$ as an example to elaborate on the details of our encoder. The superscript m is omitted for the sake of notation simplicity.

Firstly, we augment the original KG data to add a virtual self-relation r_{self} into the relation set $\mathcal{R}$, formally described as follows:

$$\mathcal{R} = \mathcal{R} \cup \{r_{self}\}. \quad (1)$$

Then, for each entity, we add a virtual triple to describe that each entity e_i has the self-relation r_{self} between itself, which is added to the triple set as follows:

$$\mathcal{T} = \mathcal{T} \cup \{\langle e_i, r_{self}, e_i \rangle \mid e_i \in \mathcal{E}\}. \quad (2)$$

To facilitate the aggregation of the GNN encoder, for entity e_i , we define the set of its neighbor relation-entity tuples as follows:

$$\mathcal{N}_i = \{\langle r_k, e_j \rangle \mid \langle e_i, r_k, e_j \rangle \in \mathcal{T}, \text{ or } \langle e_j, r_k, e_i \rangle \in \mathcal{T}\}. \quad (3)$$

Although the relations in KGs are usually directed, following most previous works, we treat them as bi-directed (or undirected). This is reasonable because the inverse of each relation is usually intuitive as well. For instance, for the triple $\langle \text{Obama}, \text{wife}, \text{Michelle} \rangle$, we can also rewrite it as: $\langle \text{Michelle}, \text{husband}, \text{Obama} \rangle$. Note that due to the addition of self-connection, entity e_i will have a special neighbor relation-entity tuple: $\langle r_{self}, e_i \rangle \in \mathcal{N}_i$.

In a layer of our KG encoder, the output representation of entity e_i is computed as follows:

$$\mathbf{h}'_i = \sigma \left(\sum_{\langle r_k, e_j \rangle \in \mathcal{N}_i} \alpha_{i,k,j} \cdot \mathbf{W}_k \cdot \mathbf{h}_j \right), \quad (4)$$

where $\mathbf{h}_j$ is the neighbor entity e_j 's feature vector output by the previous layer, $\mathbf{W}_k$ is a relation-specific projection matrix, $\alpha_{i,k,j}$ is the attention coefficient for aggregation, and σ is the non-linear activation function.

There are often thousands of relations in common KGs, and thus the relation-specific projection matrices $\mathbf{W}_k, \forall r_k \in \mathcal{R}$ may introduce too many trainable parameters, risking overfitting. Therefore, following previous studies [16, 19], we leverage the relation embedding vector $\mathbf{g}_k$ to define its related projection matrix, as follows:

$$\mathbf{W}_k = \mathbf{I} - 2 \cdot \mathbf{g}_k \cdot \mathbf{g}_k^T. \quad (5)$$

In this way, we no longer need to introduce additional parameters. In particular, the normalization constraint $\|\mathbf{g}_k\|_2 = 1$ is imposed, so that the orthogonality of $\mathbf{W}_k$ is naturally guaranteed since we can easily derive:

$$\mathbf{W}_k^T \cdot \mathbf{W}_k = (\mathbf{I} - 2 \cdot \mathbf{g}_k \cdot \mathbf{g}_k^T)^T (\mathbf{I} - 2 \cdot \mathbf{g}_k \cdot \mathbf{g}_k^T) = \mathbf{I}. \quad (6)$$

The orthogonality of $\mathbf{W}_k$ is proven to be very beneficial for the EA task [16, 19].

The attention coefficient is computed as follows. Given a neighbor tuple $\langle r_k, e_j \rangle \in \mathcal{N}_i$ of entity e_i , we first compute the proximity among the head entity e_i , the relation r_k , and the tail entity e_j :

$$\beta_{i,k,j} = \sigma(\mathbf{a}_h^T \cdot \mathbf{h}_i + \mathbf{a}_r^T \cdot \mathbf{g}_k + \mathbf{a}_t^T \cdot \mathbf{W}_k \cdot \mathbf{h}_j), \quad (7)$$

where $\mathbf{a}_h$, $\mathbf{a}_r$, and $\mathbf{a}_t$ are learnable parameter vectors for head entities, relations, and tail entities, respectively. Then, we compute the attention coefficient $\alpha_{i,k,j}$ by normalizing $\beta_{i,k,j}$ across all the elements of $\mathcal{N}_i$:

$$\alpha_{i,k,j} = \frac{\exp(\beta_{i,k,j})}{\sum_{\langle r_x, e_y \rangle \in \mathcal{N}_i} \exp(\beta_{i,x,y})}. \quad (8)$$

Note that by adding the self-relation r_{self} , each entity e_i itself acts as one of its own neighbors (Cf. Eqs. (1, 2)). In addition, we use the attention mechanism to aggregate all the neighbors (Cf. Eqs. (4, 8)). Thus, as proven by [46], our encoder is able to automatically learn the importance of arbitrary hops of neighborhood information. This is more flexible than [16, 19], which directly concatenates all hops of neighborhood information.

Recall that the encoder is shared among all the KGs, and thus, we can obtain the embeddings of the entities from every KG.

4.2 Training for Alignment

In order to pull equivalent entities together in a unified embedding space, we adopt the following margin ranking loss as the training loss of our MultiEA model:

$$L = \sum_{l \in \mathcal{S}} \sum_{l' \in \mathcal{S}'} \max(d(l) - d(l') + \lambda, 0), \quad (9)$$

where $\mathcal{S}$ is the set of positive examples, i.e., ground-truth alignment labels, $\mathcal{S}'$ is the set of negative examples, i.e., randomly generated false alignment labels, $d(\cdot)$ is a distance function, and $\lambda > 0$ is a margin hyperparameter for separating positive and negative examples.

The negative examples in $\mathcal{S}'$ are generated as follows. For each positive example $l = (e^{(1)}, e^{(2)}, \dots, e^{(m)}, \dots, e^{(M)}) \in \mathcal{S}$, we fix one entity and replace the other entities by randomly sampling from their corresponding KGs, formally described as follows:

$$\eta \times \left\{ \begin{array}{l} (e^{(1)}, \tilde{e}^{(2)}, \dots, \tilde{e}^{(m)}, \dots, \tilde{e}^{(M)}), \\ (\tilde{e}^{(1)}, e^{(2)}, \dots, \tilde{e}^{(m)}, \dots, \tilde{e}^{(M)}), \\ \dots, \\ (\tilde{e}^{(1)}, \tilde{e}^{(2)}, \dots, e^{(m)}, \dots, \tilde{e}^{(M)}), \\ \dots, \\ (\tilde{e}^{(1)}, \tilde{e}^{(2)}, \dots, \tilde{e}^{(m)}, \dots, e^{(M)}) \end{array} \right\}, \quad (10)$$

where $\tilde{e}^{(m)}$ is randomly sampled entity from $\mathcal{E}^{(m)}$ to replace the original entity $e^{(m)}$. Here, η is an integer, indicating that for each positive example, we generate η groups of negative examples in this way. Thus, each positive example corresponds to $\eta \times M$ negative examples.

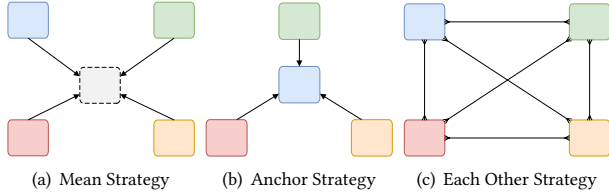


Figure 2: (a) Moving toward the mean (the central grey square with dashed line); (b) Moving toward the anchor (the central blue square); (c) Moving toward each other.

In this work, we propose three strategies for minimizing the distance among equivalent entities. The ideas are intuitively shown in Fig. 2. For a positive example $l = (e^{(1)}, e^{(2)}, \dots, e^{(m)}, \dots, e^{(M)}) \in \mathcal{S}$, we firstly obtain its associated entity embeddings from the KG encoder and denote them as: $(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}, \dots, \mathbf{h}^{(m)}, \dots, \mathbf{h}^{(M)})$. Then, the distance $d(l)$ is minimized by three strategies as follows.

(1) Moving toward Mean. This strategy lets all the equivalent entities approach their mean, as illustrated in Fig. 2(a). We first compute the mean as follows:

$$\mathbf{u} = \frac{1}{M} \cdot \sum_{m=1}^M \mathbf{h}^{(m)}. \quad (11)$$

Then, we sum the Euclidean distances between each entity embedding and the mean:

$$d(l) = \sum_{m=1}^M \|\mathbf{h}^{(m)} - \mathbf{u}\|_2. \quad (12)$$

(2) Moving toward Anchor. In practice, the candidate KGs are usually unbalanced due to various realistic factors. For instance, on our constructed multi-lingual KG dataset DBP-4, the English KG contains many more triples than the other three KGs (Cf. Table 1). Therefore, as shown in Fig. 2(b), we designate entities from KG DBP-4: en as anchor entities and let the equivalent entities from the other KGs approach these anchor entities. This idea is formally described as follows:

$$d(l) = \sum_{m=1}^M \|\mathbf{h}^{(m)} - \mathbf{h}^{(a)}\|_2, \quad (13)$$

where $\mathbf{h}^{(a)}$ is embedding of the anchor entity¹.

(3) Moving toward Each Other. This strategy lets equivalent entities approach each other. As illustrated in Fig. 2(c), we minimize the distances between all possible pairs of equivalent entities, formally described as follows:

$$d(l) = \sum_{m_1 \neq m_2} \|\mathbf{h}^{(m_1)} - \mathbf{h}^{(m_2)}\|_2, \quad (14)$$

where m_1 and m_2 are in the set $\{1, 2, \dots, m, \dots, M\}$.

4.3 Time Complexity

The time complexity of MultiEA is analyzed as follows. Let $|\mathcal{E}|$, $|\mathcal{R}|$, and $|\mathcal{T}|$ denote the total numbers of the entities, the relations, and the triples of all the M candidate KGs, respectively. Let d denote the dimensionality of entity embeddings and relation embeddings. Let $|\mathcal{S}|$ denote the number of alignment labels. Recall that each label corresponds to $\eta \times M$ negative examples.

In the KG encoding phase (Cf. Section 4.1), the time complexity involves three main computational operations. First, the aggregation operation described by Eq. (4) has time complexity $O(|\mathcal{E}| \cdot d^2)$. Second, the computation of relation-specific projection matrices described by Eq. (5) has time complexity $O(|\mathcal{R}| \cdot d^2)$. Finally, as described by Eqs. (7, 8), the computation of attention coefficients has time complexity $O(|\mathcal{T}| \cdot (d + d + d^2))$. Considering that in practice, the embedding dimensionality d is a relatively small value, the time complexity of KG encoding is $O(|\mathcal{E}| + |\mathcal{R}| + |\mathcal{T}|)$.

In the alignment phase (Cf. Section 4.2), as described by Eq. (12) and Eq. (13), both the mean strategy and the anchor strategy require time complexity $O(M)$. As described by Eq. (14), the each other strategy requires time complexity $O(M^2)$. As described by Eq. (9), taking the number of labels into consideration. The mean strategy and the anchor strategy have time complexity of $O(|\mathcal{S}| \cdot \eta \cdot M^2)$, and each other strategy has time complexity $O(|\mathcal{S}| \cdot \eta \cdot M^3)$.

In practice, $|\mathcal{S}|$, η , and M are much smaller than $|\mathcal{E}|$, $|\mathcal{R}|$, and $|\mathcal{T}|$. Therefore, the overall time complexity for training MultiEA is equal to $O(|\mathcal{E}| + |\mathcal{R}| + |\mathcal{T}|)$.

¹Note that when $\mathbf{h}^{(m)}$ is designated as anchor entity, then $\|\mathbf{h}^{(m)} - \mathbf{h}^{(a)}\|_2 = 0$.

4.4 Inference Enhancement

After training the model, we can obtain the embeddings of the entities from all the KGs. Based on these embeddings, we can compute the similarity between any two entities that are from different KGs. For example, given entity $e_i^{(m_1)} \in \mathcal{E}^{(m_1)}$, and entity $e_j^{(m_2)} \in \mathcal{E}^{(m_2)}$, we compute the similarity between them as follows²:

$$S_{i,j}^{m_1-m_2} = 1 - \frac{\|\mathbf{h}_i^{(m_1)} - \mathbf{h}_j^{(m_2)}\|_2}{2}. \quad (15)$$

Thus, we can obtain a matrix $S^{m_1-m_2}$ to describe the entity similarities between $\mathcal{G}^{(m_1)}$ and $\mathcal{G}^{(m_2)}$. Finally, the alignment task is accomplished based on the similarities among all KG pairs.

It is worth noting that we find that Eq. (15) only captures the first-order similarity, which may not be sufficient since some higher-order similarity information is ignored. Therefore, in this work, we propose to further enhance the similarity matrix by incorporating higher-order similarities, which can be composed by the matrix product operation. For example, if there are three KGs, $\mathcal{G}^{(m_1)}$, $\mathcal{G}^{(m_2)}$, and $\mathcal{G}^{(m_3)}$, we can enhance the similarity matrix $S^{m_1-m_2}$ as follows:

$$\tilde{S}^{m_1-m_2} = \gamma_1 \cdot S^{m_1-m_2} + \gamma_2 \cdot S^{m_1-m_3} \cdot S^{m_3-m_2}, \quad (16)$$

where γ_1 and γ_2 are hyperparameters to balance the two terms. They are real numbers in the range $[0, 1]$ and satisfy $\gamma_1 + \gamma_2 = 1$. Similarly, if there are four KGs, we can enhance $S^{m_1-m_2}$ as follows:

$$\begin{aligned} \tilde{S}^{m_1-m_2} = & \gamma_1 \cdot S^{m_1-m_2} + \gamma_2 \cdot S^{m_1-m_3} \cdot S^{m_3-m_2} \\ & + \gamma_3 \cdot S^{m_1-m_4} \cdot S^{m_4-m_2}, \end{aligned} \quad (17)$$

where γ_1 , γ_2 , and γ_3 are hyperparameters in the range $[0, 1]$ and satisfy $\gamma_1 + \gamma_2 + \gamma_3 = 1$.

Here, we only incorporate the two-order similarities by the matrix product operation. In practice, higher-order similarities can be similarly incorporated through more matrix product operations. Finally, we can identify equivalent entities according to the enhanced similarity matrix. In this way, the model can utilize more comprehensive information, helping improve the inference performance.

5 Experiment

In this section, we conduct extensive experiments to verify the effectiveness of our MultiEA. The key experimental setting details are as follows.

Datasets. As far as we know, there is no existing EA benchmark dataset that can support the task of aligning multiple KGs. To this end, we construct two novel benchmark datasets, i.e., DBP-4 and DWY-3, based on previously widely used real-world datasets [25, 27]. DBP-4 contains four candidate KGs, and DWY-3 contains three candidate KGs to be aligned. Their key statistics are listed in Table 1. Please see Appendix A.2 for more details about them.

Evaluation Metric. Traditional pair-wise KG alignment methods generally use *Hits@K* to evaluate the alignment performance, but it does not work for our task of multiple KG alignment. Hence, in this work, in the spirit of *Hits@K*, we extend *Hits@K* to a novel

²Since entity embeddings are normalized, i.e., $\|\mathbf{h}_i\|_2 = 1, \forall i$, the computed similarities lie in the range $[0, 1]$. Thus, we can compose high-order similarities by matrix product operation later.

Table 1: Dataset statistics.

Datasets	KGs	$ \mathcal{E} $	$ \mathcal{R} $	$ \mathcal{T} $	$ \mathcal{S} $
DBP-4	en	8901	1034	77483	2539
	fr	3545	774	15843	
	ja	4326	519	32427	
	zh	3893	619	21497	
DWY-3	dbp	23784	246	94985	20729
	wiki	22839	153	92019	
	yago	22063	30	77457	

evaluation metric, i.e., *M-Hits@K*, which is more challenging than the traditional metric *Hits@K*, and can better reflect the model performance of aligning multiple KGs. Please see Appendix A.3 for the detailed definitions.

Baselines and MultiEA Variants. We select six representative supervised EA baselines that only leverage the structural information of candidate KGs, including two Trans-based EA methods MTransE [5] and IPTransE [54], and four GNN-based EA methods GCN-Align [40], NAEA [55], KECG [11], PSR [16]. We also set six variants for our MultiEA by considering all the possible combinations of alignment strategies and the inference enhancement operation. Please see Appendix A.4 for more details of baselines and MultiEA variants.

Implementation Details. Please see Appendix A.5 for the detailed implementation details, and see Appendix A.6 for sensitivity studies of four key hyperparameters of MultiEA.

5.1 Multiple KG Alignment

We first quantitatively evaluate the effectiveness of our MultiEA in aligning multiple (more than two) KGs. Specifically, we apply all six baseline methods as well as all six variants of MultiEA to the two constructed benchmark datasets, and use the metric *M-Hits@K* newly defined in Eq. (22) to compare their accuracy. The results are reported in Table 2.

Comparison with Baselines. It is worth noting that we cannot directly apply baseline EA methods to the multiple KG alignment task since they are specifically designed for the traditional pair-wise KG alignment task. To make them compatible, we split the concerned task into multiple pair-wise sub-tasks and separately align each pair of KGs. To reflect this, baselines are marked by * in the table. We can see that our MultiEA (especially the three variants with the inference enhancement module at the bottom of the table) can significantly outperform all the baselines on the two datasets. This may be due to the fact that our MultiEA can effectively capture the global alignment information by projecting the entities of all the candidate KGs in a unified embedding space. Besides, GNN-based EA baselines show better performance than Trans-based EA baselines. This is consistent with the findings of many previous studies [11, 16, 40, 55].

Abalation Studies. As we can see in Table 2, our variant MultiEA+each+infer achieves the best overall results in most cases,

Table 2: Comparison of multiple KG alignment accuracy. The best results are bolded, and the second-best results are underlined. The notation * denotes that we re-implement the baseline methods to make them compatible with the task (the same below).

Methods	DBP-4			DWY-3		
	<i>M-Hits@1</i>	<i>M-Hits@10</i>	<i>M-Hits@20</i>	<i>M-Hits@1</i>	<i>M-Hits@10</i>	<i>M-Hits@20</i>
MTransE*	3.40%	19.72%	32.64%	25.83%	55.94%	65.31%
IPTransE*	4.11%	23.36%	39.55%	28.74%	60.02%	68.84%
GCN-Align*	4.48%	27.62%	44.94%	32.46%	63.18%	73.91%
KECG*	5.62%	33.35%	49.61%	34.08%	67.51%	77.52%
NAEA*	6.14%	34.13%	48.07%	35.17%	69.88%	79.43%
PSR*	7.84%	38.17%	50.97%	37.55%	71.85%	80.13%
MultiEA+mean-infer	5.95%	40.48%	51.02%	36.68%	69.33%	79.65%
MultiEA+anchor-infer	6.32%	40.33%	52.08%	37.54%	70.21%	79.19%
MultiEA+each-infer	6.17%	41.70%	52.00%	36.76%	71.52%	79.18%
MultiEA+mean+infer	10.03%	48.18%	59.89%	40.54%	73.90%	81.24%
MultiEA+anchor+infer	10.58%	<u>48.80%</u>	<u>59.95%</u>	42.33%	<u>74.13%</u>	<u>83.54%</u>
MultiEA+each+infer	<u>10.25%</u>	50.38%	61.75%	<u>41.19%</u>	75.44%	85.42%

Ali_Khamenei	阿里 哈梅内伊	Ali_Khamenei	アリー・ハーメネー
American_Civil_War	南北戦争	Guerre_de_Sécession	南北戦争
McGill_University	麦吉尔大学	Université_McGill	マギル大学
Ferdinand_I_of_Austria	斐迪南一世_(奥地利)	Ferdinand_Ier_d'Autriche	フェルディナント1世_(オーストリア皇帝)
Yukon	育空	Yukon	ユーコン準州
Imagine_(John_Lennon_album)	想像_(专辑)	Imagine_(album_de_John_Lennon)	イマジン_(アルバム)
Frederick_VIII_of_Denmark	弗雷德里克八世	Frédéric_VIII_de_Danemark	フレゼリク8世_(デンマーク王)
Maurice_Gibb	莫里斯·吉布	Maurice_Gibb	モーリス・ギブ
European_People's_Party	歐洲人民黨	Parti_populaire_européen	欧州人民党
Nicholas_II_of_Russia	尼古拉二世_(俄罗斯)	Nicolas_II	ニコライ2世

DBP-4: en DBP-4: zh DBP-4: fr DBP-4: ja

Figure 3: The entity names of the ten groups of equivalent entities discovered by MultiEA on DBP-4.

Popular_Democratic_Party_(Puerto_Rico)	Popular Democratic Party	Popular_Democratic_Party_(Puerto_Rico)
Forlì	Forlì	Forlì
Idigh	Idigh	Idigh
Bradley_County,_Tennessee	Bradley County	Bradley_County,_Tennessee
The_Specialist	The Specialist	The_Specialist
Oskar_Perron	Oskar Perron	Oskar_Perron
Dakota_Wesleyan_University	Dakota Wesleyan University	Dakota_Wesleyan_University
Carmen_Yul_fí_Cruz	Carmen Yul fí Cruz	Carmen_Yul_fí_Cruz
Charles_Loewner	Charles Loewner	Charles_Loewner
Grand_Duke_Konstantin_Konstantinovich_of_Russia	Grand Duke Konstantin Konstantinovich of Russia	Grand_Duke_Konstantin_Konstantinovich_of_Russia

DWY-3: dbp DWY-3: wiki DWY-3: yago

Figure 4: The entity names of ten groups of equivalent entities discovered by MultiEA on DWY-3. Note that, for DWY-3: wiki, the original data is in the form of the URLs of the entities. We show the corresponding entries of the involved URLs.

showing its superior effectiveness in aligning multiple KGs. Overall, MultiEA+anchor+infer and MultiEA+mean+infer achieve the second-best and the third-best results, respectively. The situation is the same for the other three variants without the inference enhancement module. Therefore, we recommend the “each other”

strategy as the default strategy for most cases. If users prefer higher efficiency and there is an obvious anchor KG, we recommend the “anchor” strategy, and otherwise, we recommend the “mean” strategy. From another dimension, we can observe that all three variants with the inference enhancement module significantly outperform

the other three variants without this module. This indicates that the inference enhancement module is significantly beneficial for this task.

5.2 Case Study

To intuitively show the effectiveness of our MultiEA, in Fig. 3 and Fig. 4, we list ten groups of equivalent entities discovered by MultiEA (the best-performing variant MultiEA+*each+infer*) on the two datasets. As we can see, MultiEA can effectively discover equivalent entities across all the candidate KGs.

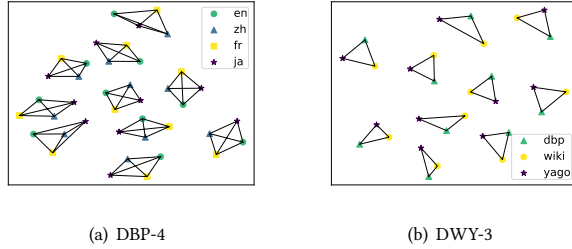


Figure 5: Visualization of the embeddings of the ten groups of equivalent entities listed in Fig. 3 and Fig. 4.

Further, we visualize the embeddings of these entities. Specifically, we utilize the t-SNE algorithm [15] to project them into the 2-dimensional Euclidean space. Fig. 5(a) and Fig. 5(b) show the results on DBP-4 and DWY-3, respectively, where different colors and different shapes mark the entities from different KGs, and the equivalent entities are connected by black lines. As we can see, on both datasets, the ten groups of equivalent entities are obviously gathered together, indicating the effectiveness of the alignment mechanism of MultiEA. On the other hand, a proper margin is maintained between different groups, avoiding the risk of model overfitting. The result on DWY-3 shows a relatively better distribution. This is consistent with the higher accuracy on DWY-3 as reported in Table 2.

5.3 Efficiency Study

In this experiment, we demonstrate the efficiency advantage of our MultiEA in aligning multiple KGs. As shown in Table 3 and Table 4, we first report the experimental results of our re-implemented GNN-based EA baselines as described in Section 5.1. That is, they align multiple candidate KGs by splitting this task into multiple separate pair-wise sub-tasks, as marked by * in the tables. Then, we apply the “each other” alignment strategy and the inference enhancement module of our MultiEA+*each+infer* to each baseline X, as denoted as MultiEA (X) in the tables.

In addition to the $M\text{-Hits}@K$ scores, we report the number of model parameters, the memory, and the time required by each method during its training. As we can see, our MultiEA can help each baseline achieve significantly better performance. Moreover, MultiEA can help each baseline significantly reduce the requirements of parameters, space, and time resources. This demonstrates the superior efficiency of our MultiEA.

5.4 Pair-wise KG Alignment

We also apply our MultiEA (the variant MultiEA+*each+infer*) to the traditional pair-wise KG alignment task. Firstly, based on the two constructed benchmark datasets, we induce several pair-wise KG alignment sub-datasets. For DBP-4, we induce three sub-datasets, i.e., DBP-4: en-fr, DBP-4: en-ja, and DBP-4: en-zh. For DWY-3, we induce two sub-datasets, i.e., DWY-3: dbp-wiki and DWY-3: dbp-yago. For baselines, we reproduce their results on our induced datasets. Then, we re-implement the encoder of MultiEA as that used in a baseline X, denoted as MultiEA (X). This can eliminate the encoder’s influence on the alignment results, and thus investigate whether our proposed “each other” strategy and inference enhancement module help improve the performance of baselines on this task. By convention, we use the widely used metric $Hits@K$ described by Eq. (19) to evaluate the alignment accuracy.

Table 5 shows the final $Hits@1$ scores (see Appendix A.7 for $Hits@10$ scores). In addition to the original experimental results, we also compute the relative gain to show the improvement more intuitively. As we can see, our MultiEA can always help improve the baselines’ performance on the traditional pair-wise KG alignment task. Besides, we have some other findings as follows. PSR, KECG, and NAEA perform better than GCN-Align, indicating the effectiveness of the attention mechanism used in their KG encoders. Moreover, MultiEA (PSR), MultiEA (KECG), and MultiEA (NAEA) outperform MultiEA (GCN-Align). This demonstrates that our MultiEA framework can effectively exploit the structural information captured by the graph attention mechanisms. Two Trans-based methods, MTransE and IPTransE, show worse performance than GNN-based methods, which is consistent with the conclusions of previous works [11, 40, 55].

6 Conclusion

In this work, we note that existing methods mainly focus on aligning only a pair of candidate KGs, ignoring the multiplicity of the candidate KGs to be aligned. To fill the research gap in this field, we formulate a novel problem of aligning multiple (more than two) KGs and propose the MultiEA framework to effectively solve this problem. To enhance the inference performance, we innovatively propose to incorporate higher-order similarities. Last but not least, we construct two new benchmark datasets and define the evaluation metric for the problem. We have conducted extensive experiments, and the results show that our MultiEA framework can effectively and efficiently align multiple KGs in a single pass. In addition, MultiEA can improve the performance of traditional pair-wise KG alignment. We will make our source codes and the constructed benchmark datasets publicly available to facilitate further research on this problem, which we believe will inspire more interesting works in the future.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grants 62303366, 62425605, and 62133012, and in part by the Key Research and Development Program of Shaanxi under Grants 2025CY-YBXM-041, 2022ZDLGY01-10, and 2024CY2-GJHX-15.

Table 3: Comparison of accuracy and efficiency on DBP-4.

Methods	<i>M-Hits@1</i>	<i>M-Hits@10</i>	<i>M-Hits@20</i>	Parameter	Memory	Time
GCN-Align*	4.48%	27.62%	44.94%	10.04M	38.30MB	382.77s
MultiEA (GCN-Align)	7.32%	43.75%	54.14%	5.35M	20.43MB	145.49s
KECG*	5.62%	33.35%	49.61%	10.24M	39.07MB	478.30s
MultiEA (KECG)	8.42%	45.93%	57.60%	5.42M	20.68MB	185.66s
NAEA*	6.14%	34.14%	48.07%	12.31M	46.95MB	522.60s
MultiEA (NAEA)	8.95%	46.15%	57.82%	6.43M	24.56MB	206.13s
PSR*	7.84%	38.17%	50.97%	11.13M	42.47MB	1336.05s
MultiEA (PSR)	9.98%	49.13%	61.30%	6.05M	23.06MB	576.19s

Table 4: Comparison of accuracy and efficiency on DWY-3.

Methods	<i>M-Hits@1</i>	<i>M-Hits@10</i>	<i>M-Hits@20</i>	Parameter	Memory	Time
GCN-Align*	32.46%	63.18%	73.91%	23.80M	90.80MB	566.43s
MultiEA (GCN-Align)	36.75%	69.75%	80.26%	17.78M	67.83MB	315.44s
KECG*	34.08%	67.51%	77.52%	24.04M	91.66MB	780.29s
MultiEA (KECG)	38.52%	71.85%	82.97%	17.78M	67.83MB	407.16s
NAEA*	35.17%	69.88%	79.43%	25.63M	93.96MB	868.15s
MultiEA (NAEA)	40.66%	74.69%	83.66%	18.31M	69.84MB	480.61s
PSR*	37.55%	71.85%	80.13%	23.85M	90.98MB	1974.62s
MultiEA (PSR)	41.68%	74.76%	84.72%	17.58M	67.07MB	804.37s

Table 5: The *Hits@1* scores of pair-wise KG alignment.

Methods	DBP-4			DWY-3	
	en-zh	en-fr	en-ja	dbp-wiki	dbp-yago
MTransE	14.63%	15.91%	13.54%	35.16%	44.35%
MultiEA (MTransE)	15.29%	16.05%	15.68%	36.05%	46.17%
Relative Gain ↑	+4.51%	+0.88%	+15.81%	+2.53%	+4.10%
IPTransE	19.84%	20.95%	21.38%	43.51%	55.19%
MultiEA (IPTransE)	20.26%	21.01%	23.29%	43.68%	57.06%
Relative Gain ↑	+0.11%	+0.33%	+8.93%	+0.39%	+3.38%
GCN-Align	21.48%	24.01%	21.54%	44.75%	58.63%
MultiEA (GCN-Align)	21.57%	25.71%	22.19%	45.48%	60.08%
Relative Gain ↑	+0.42%	+7.08%	+3.02%	+1.63%	+2.47%
KECG	25.71%	27.44%	26.77%	47.15%	61.97%
MultiEA (KECG)	26.14%	29.48%	27.49%	50.02%	62.44%
Relative Gain ↑	+1.67%	+7.43%	+2.69%	+6.09%	+0.76%
NAEA	24.69%	26.92%	26.35%	46.51%	61.37%
MultiEA (NAEA)	25.75%	28.16%	26.95%	49.18%	61.52%
Relative Gain ↑	+4.29%	+4.61%	+2.28%	+5.74%	+0.24%
PSR	25.59%	28.20%	27.87%	48.23%	60.52%
MultiEA (PSR)	26.27%	30.18%	28.03%	51.36%	62.14%
Relative Gain ↑	+2.65%	+7.02%	+0.57%	+6.49%	+3.12%

References

- [1] Mehdi Ali, Max Berrendorf, Charles Tapley Hoyt, Laurent Vermue, Mikhail Galkin, Sahand Sharifzadeh, Asja Fischer, Volker Tresp, and Jens Lehmann. 2022. Bringing Light Into the Dark: A Large-Scale Evaluation of Knowledge Graph Embedding Models Under a Unified Framework. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 12 (2022), 8825–8845.
- [2] Jonathan T Barron. 2017. Continuously differentiable exponential linear units. *arXiv preprint arXiv:1704.07483* (2017).
- [3] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems* 26 (2013).
- [4] Yixin Cao, Zhiyuan Liu, Chengjiang Li, Juanzi Li, and Tat-Seng Chua. 2019. Multi-Channel Graph Neural Network for Entity Alignment. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 1452–1461.
- [5] Muhao Chen, Yingtao Tian, Mohan Yang, and Carlo Zaniolo. 2016. Multilingual knowledge graph embeddings for cross-lingual knowledge alignment. *arXiv preprint arXiv:1611.03954* (2016).
- [6] Congcong Ge, Xiaozhe Liu, Lu Chen, Yunjun Gao, and Baihua Zheng. 2021. LargeEA: aligning entities for large-scale knowledge graphs. *Proceedings of the VLDB Endowment* 15, 2 (2021), 237–245.
- [7] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*. 249–256.
- [8] Yanchao Hao, Yuanzhe Zhang, Shizhu He, Kang Liu, and Jun Zhao. 2016. A joint embedding method for entity alignment of knowledge bases. In *Knowledge Graph and Semantic Computing: Semantic, Knowledge, and Linked Big Data: First China Conference, CCKS 2016, Beijing, China, September 19–22, 2016, Revised Selected Papers 1*. 3–14.
- [9] Yuncheng Hua, Yuan-Fang Li, Gholamreza Haffari, Guilin Qi, and Tongtong Wu. 2020. Few-Shot Complex Knowledge Base Question Answering via Meta Reinforcement Learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 5827–5837.
- [10] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [11] Chengjiang Li, Yixin Cao, Lei Hou, Jiaxin Shi, Juanzi Li, and Tat-Seng Chua. 2019. Semi-supervised entity alignment via joint knowledge embedding model and cross-graph model. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.
- [12] Qian Li, Shu Guo, Yangyifei Luo, Cheng Ji, Lihong Wang, Jiawei Sheng, and Jianxin Li. 2023. Attribute-Consistent Knowledge Graph Representation Learning for Multi-Modal Entity Alignment. In *Proceedings of the ACM Web Conference 2023*. 2499–2508.
- [13] Yankai Lin, Zhiyuan Liu, Huanbo Luan, Maosong Sun, Siwei Rao, and Song Liu. 2015. Modeling relation paths for representation learning of knowledge bases. *arXiv preprint arXiv:1506.00379* (2015).
- [14] Xiao Liu, Haoyun Hong, Xinghao Wang, Zeyi Chen, Evgeny Kharlamov, Yuxiao Dong, and Jie Tang. 2022. Selfkg: Self-supervised entity alignment in knowledge graphs. In *Proceedings of the ACM Web Conference 2022*. 860–870.
- [15] Laurens van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *JMLR* 9, Nov (2008), 2579–2605.
- [16] Xin Mao, Wenting Wang, Yuanbin Wu, and Man Lan. 2021. Are negative samples necessary in entity alignment? an approach with high performance, scalability and robustness. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 1263–1273.
- [17] Xin Mao, Wenting Wang, Yuanbin Wu, and Man Lan. 2022. Lightea: A scalable, robust, and interpretable entity alignment framework via three-view label propagation. *arXiv preprint arXiv:2210.10436* (2022).
- [18] Xin Mao, Wenting Wang, Huimin Xu, Man Lan, and Yuanbin Wu. 2020. MRAEA: an efficient and robust entity alignment approach for cross-lingual knowledge graph. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 420–428.
- [19] Xin Mao, Wenting Wang, Huimin Xu, Yuanbin Wu, and Man Lan. 2020. Relational reflection entity alignment. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 1095–1104.
- [20] Xin Mu, Feida Zhu, Ee-Peng Lim, Jing Xiao, Jianzong Wang, and Zhi-Hua Zhou. 2016. User identity linkage by latent user space modelling. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1775–1784.
- [21] Hao Nie, Xianpei Han, Le Sun, Chi Man Wong, Qiang Chen, Suhui Wu, and Wei Zhang. 2021. Global structure and local semantics-preserved embeddings for entity alignment. In *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*. 3658–3664.
- [22] Apoorv Saxena, Aditya Tripathi, and Partha Talukdar. 2020. Improving multi-hop question answering over knowledge graphs using knowledge base embeddings. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 4498–4507.
- [23] Sen Su, Li Sun, Zhongbao Zhang, Gen Li, and Jielun Qu. 2018. MASTER: across Multiple social networks, integrate Attribute and STructure Embedding for Recommendation. In *IJCAI*, Vol. 18. 3863–3869.
- [24] Li Sun, Zhongbao Zhang, Gen Li, Pengxin Ji, Sen Su, and Philip S Yu. 2023. MC2: Unsupervised Multiple Social Network Alignment. *ACM Transactions on Intelligent Systems and Technology* (2023).
- [25] Zequn Sun, Wei Hu, and Chengkai Li. 2017. Cross-lingual entity alignment via joint attribute-preserving embedding. In *The Semantic Web–ISWC 2017: 16th International Semantic Web Conference, Vienna, Austria, October 21–25, 2017, Proceedings, Part I* 16. 628–644.
- [26] Zequn Sun, Wei Hu, Chengming Wang, Yuxin Wang, and Yuzhong Qu. 2022. Revisiting embedding-based entity alignment: A robust and adaptive method. *IEEE Transactions on Knowledge and Data Engineering* 35, 8 (2022), 8461–8475.
- [27] Zequn Sun, Wei Hu, Qingheng Zhang, and Yuzhong Qu. 2018. Bootstrapping entity alignment with knowledge graph embedding. In *IJCAI*, Vol. 18.
- [28] Zequn Sun, Jiacheng Huang, Wei Hu, Muhao Chen, Lingbing Guo, and Yuzhong Qu. 2019. Transedge: Translating relation-contextualized embeddings for knowledge graphs. In *The Semantic Web–ISWC 2019: 18th International Semantic Web Conference, Auckland, New Zealand, October 26–30, 2019, Proceedings, Part I* 18. 612–629.
- [29] Zequn Sun, Chengming Wang, Wei Hu, Muhao Chen, Jian Dai, Wei Zhang, and Yuzhong Qu. 2020. Knowledge graph alignment network with gated multi-hop neighborhood aggregation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 222–229.
- [30] Zequn Sun, Qingheng Zhang, Wei Hu, Chengming Wang, Muhao Chen, Farahnaz Akrami, and Chengkai Li. 2020. A Benchmarking Study of Embedding-based Entity Alignment for Knowledge Graphs. *Proceedings of the VLDB Endowment* 13, 11 (2020).
- [31] Nguyen Thanh Tam, Huynh Thanh Trung, Hongzhi Yin, Tong Van Vinh, Darnbi Sakong, Bolong Zheng, and Nguyen Quoc Viet Hung. 2020. Entity alignment for knowledge graphs with multi-order convolutional networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 9 (2020), 4201–4214.
- [32] Sinan Tan, Mengmeng Ge, Di Guo, Huaping Liu, and Fuchun Sun. 2023. Knowledge-Based Embodied Question Answering. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 10 (2023), 11948–11960.
- [33] Wei Tang, Fenglong Su, Haifeng Sun, Qi Qi, Jingyu Wang, Shimin Tao, and Hao Yang. 2023. Weakly Supervised Entity Alignment with Positional Inspiration. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 814–822.
- [34] Xiaobin Tang, Jing Zhang, Bo Chen, Yang Yang, Hong Chen, and Cuiping Li. 2020. BERT-INT: A BERT-based interaction model for knowledge graph alignment. *interactions* 100 (2020), e1.
- [35] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [36] Hongwei Wang, Miao Zhao, Xing Xie, Wenjie Li, and Minyi Guo. 2019. Knowledge graph convolutional networks for recommender systems. In *The world wide web conference*. 3307–3313.
- [37] Mengzhu Wang, Junyang Chen, Ye Wang, Shanshan Wang, Hao Su, Zhiguo Gong, Kaishun Wu, Zhenghan Chen, et al. 2023. Joint adversarial domain adaptation with structural graph alignment. *IEEE Transactions on Network Science and Engineering* 11, 1 (2023), 604–612.
- [38] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. 2017. Knowledge graph embedding: A survey of approaches and applications. *IEEE transactions on knowledge and data engineering* 29, 12 (2017), 2724–2743.
- [39] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 950–958.
- [40] Zhichun Wang, Qingsong Lv, Xiaohan Lan, and Yu Zhang. 2018. Cross-lingual knowledge graph alignment via graph convolutional networks. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 349–357.
- [41] Yuting Wu, Xiao Liu, Yansong Feng, Zheng Wang, Rui Yan, and Dongyan Zhao. 2019. Relation-aware entity alignment for heterogeneous knowledge graphs. *arXiv preprint arXiv:1908.08210* (2019).
- [42] Yuting Wu, Xiao Liu, Yansong Feng, Zheng Wang, and Dongyan Zhao. 2019. Jointly learning entity and relation representations for entity alignment. *arXiv preprint arXiv:1909.09317* (2019).
- [43] Yuting Wu, Xiao Liu, Yansong Feng, Zheng Wang, and Dongyan Zhao. 2020. Neighborhood Matching Network for Entity Alignment. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 6477–6487.
- [44] Kun Xu, Liwei Wang, Mo Yu, Yansong Feng, Yan Song, Zhiguo Wang, and Dong Yu. 2019. Cross-lingual Knowledge Graph Alignment via Graph Matching Neural Network. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 3156–3161.
- [45] Hsiu-Wei Yang, Yanyan Zou, Peng Shi, Wei Lu, Jimmy Lin, and Xu Sun. 2019. Aligning Cross-Lingual Entities with Multi-Aspect Information. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and*

- the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). 4431–4441.
- [46] Yaming Yang, Ziyu Guan, Jianxin Li, Wei Zhao, Jiangtao Cui, and Quan Wang. 2021. Interpretable and efficient heterogeneous graph convolutional network. *TKDE* (2021).
- [47] Rui Ye, Xin Li, Yujie Fang, Hongyu Zang, and Mingzhong Wang. 2019. A vectorized relational graph convolutional network for multi-relational network alignment. In *IJCAI*. 4135–4141.
- [48] Kaisheng Zeng, Chengjiang Li, Lei Hou, Juanzi Li, and Ling Feng. 2021. A comprehensive survey of entity alignment for knowledge graphs. *AI Open* 2 (2021), 1–13.
- [49] Weixin Zeng, Xiang Zhao, Zhen Tan, Jiuyang Tang, and Xueqi Cheng. 2023. Matching knowledge graphs in entity embedding spaces: an experimental study. *IEEE Transactions on Knowledge and Data Engineering* 35, 12 (2023), 12770–12784.
- [50] Qingheng Zhang, Zequn Sun, Wei Hu, Muhao Chen, Lingbing Guo, and Yuzhong Qu. 2019. Multi-view knowledge graph embedding for entity alignment. *arXiv preprint arXiv:1906.02390* (2019).
- [51] Yutao Zhang, Jie Tang, Zhilin Yang, Jian Pei, and Philip S Yu. 2015. Cosnet: Connecting heterogeneous social networks with local and global consistency. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 1485–1494.
- [52] Xuejiao Zhao, Huanhuan Chen, Zhenchang Xing, and Chunyan Miao. 2021. Brain-inspired search engine assistant based on knowledge graph. *IEEE Transactions on Neural Networks and Learning Systems* (2021).
- [53] Xiang Zhao, Weixin Zeng, Jiuyang Tang, Wei Wang, and Fabian M Suchanek. 2020. An experimental study of state-of-the-art entity alignment approaches. *IEEE Transactions on Knowledge and Data Engineering* 34, 6 (2020), 2610–2625.
- [54] Hao Zhu, Ruobing Xie, Zhiyuan Liu, and Maosong Sun. 2017. Iterative Entity Alignment via Joint Knowledge Embeddings. In *IJCAI*, Vol. 17. 4258–4264.
- [55] Qiannan Zhu, Xiaofei Zhou, Jia Wu, Jianlong Tan, and Li Guo. 2019. Neighborhood-Aware Attentional Representation for Multilingual Knowledge Graphs. In *IJCAI*. 1943–1949.

A Appendix

A.1 More Related Work

EA Enhancement. In recent years, several works have further enhanced the EA performance by exploiting attribute information and entity name information. HMAN [45], BERT-INT [34], and ACK-MMEA [12] can leverage the rich attributes of entities. Besides, PSR [16], GMNN [44], NMN [43], HGCN-JE [42], RDGCN [41], LightEA [17], and SelfKG [14] can use the word embeddings of the entity names to effectively initialize entity embeddings, greatly improving the performance of EA.

Alignment Methods in Other Fields. In the social network analysis field, there are several methods that have made some efforts to align multiple social networks. COSNET [51] leverages multiple networks to enhance the alignment performance of two networks. It requires exponential time complexity to build a matching graph, which is impractical for many scenarios. ULink [20] learns a latent user space based on user attributes. It cannot exploit structural information, and thus it is not a strict social network alignment approach. MC² [24] combines both structural information and attribute information to infer a common base by matrix factorization. It requires all social networks to have attributes, and its time computational complexity is quadratic to the number of users. MASTER [23] embeds multiple social networks in a common latent space through collaborative matrix factorization, which also requires square computational complexity. JADA-SGA [37] develops a structural graph alignment-based method for solving the domain adaptation problem. These methods are specially designed methods for other fields, and they generally require sophisticated optimization. In summary, unlike these methods, in this work, we study the problem of aligning multiple (more than two) KGs based

on structural information, and we develop a GNN-based model to solve the problem in an end-to-end optimization manner.

A.2 More Details of Benchmark Datasets

• **DBP-4.** This dataset contains four KGs from different language sources, including English (en) KG, Chinese (zh) KG, Japanese (ja) KG, and French (fr) KG. It is constructed based on the DBP15K dataset, which was originally released by [25]. The original alignment labels describe the correspondence from English KG to the other three KGs, which can be described as: en-fr, en-zh, and en-ja. We treat the English KG as an intermediary and complement the alignment information among all four KGs, which can be described as: en-fr-zh-ja. Specifically, we filter entities from different KGs that correspond to the same English entity. Intuitively, if we have: $e^{(en)} = e^{(fr)}$, $e^{(en)} = e^{(zh)}$, and $e^{(en)} = e^{(ja)}$. Then, we can infer: $e^{(en)} = e^{(fr)} = e^{(zh)} = e^{(ja)}$, which means that all these four entities from different KGs are equivalent.

• **DWY-3.** This dataset contains three KGs, i.e., DBpedia (dbp), Wikidata (wiki), and YAGO (yago). It is constructed based on the DWY100K dataset, which was originally released by [27]. Its original alignment labels describe the correspondence from DBpedia to the other two KGs, which can be described as: dbp-wiki and dbp-yago. Similar to the construction of DBP-4, we treat the DBpedia KG as an intermediary to complement the alignment information among all three KGs, which can be described as: dbp-wiki-yago.

It is worth noting that the condition under which we construct labels is very strict. As a result, the number of labels is very small compared to the number of entities and triples. To address the issue, we further induce a smaller knowledge graph. Specifically, inspired by the previous work [25], for each entity involved in labels, we select its popular neighbor entities whose degrees are larger than a threshold (we set 15 in this work). This target entity and its high-degree neighbor entries are added to a new entity set, and the involved relations are added to a new relation set. Then, we induce a new set of triples based on the selected entities and relations.

A.3 Definition of Evaluation Metric

Traditional pair-wise KG alignment methods generally use *Hits@K* to evaluate the alignment performance. Specifically, given two KGs: $\mathcal{G}^{(l)}$ and $\mathcal{G}^{(r)}$, we first consider aligning $\mathcal{G}^{(r)}$ to $\mathcal{G}^{(l)}$, and define a temporary metric $l_Hits@K$ to measure the proportion of the entities in $\mathcal{G}^{(l)}$ whose counterpart in $\mathcal{G}^{(r)}$ rank in top-K:

$$l_Hits@K = \frac{\sum_{n=1}^N \mathbb{I}(e_n^{(r)} \text{ ranks in top-K})}{N}, \quad (18)$$

where $\mathbb{I}$ is the indicator function that outputs 1 if the input fact is true and 0 otherwise. Then, we consider aligning $\mathcal{G}^{(l)}$ to $\mathcal{G}^{(r)}$, and similarly define a temporary metric $r_Hits@K$ in the opposite direction. The final metric *Hits@K* is computed as the average of the two temporary metrics:

$$Hits@K = \frac{1}{2} \cdot (l_Hits@K + r_Hits@K). \quad (19)$$

Unfortunately, *Hits@K* does not work for our task of multiple KG alignment. Hence, in the spirit of *Hits@K*, we define a novel evaluation metric. Given M KGs: $\{\mathcal{G}^{(1)}, \mathcal{G}^{(2)}, \dots, \mathcal{G}^{(m)}, \dots, \mathcal{G}^{(M)}\}$

where $M > 2$, we treat each KG $\mathcal{G}^{(m)}$ as the target KG, and consider aligning the other $M - 1$ KGs to it. First, we count the number of entities from $\mathcal{G}^{(m)}$ whose **all** $M - 1$ counterparts from the other $M - 1$ KGs rank in top-K, described as follows:

$$C = \sum_{n=1}^N \mathbb{I}(e_n^{(1)}, \dots, e_n^{(m-1)}, e_n^{(m+1)}, \dots, e_n^{(M)} \text{ rank in top-K}), \quad (20)$$

where $\mathbb{I}(\cdot)$ is an indicator function that outputs 1 if **all** the input entities rank in top-K, and 0 otherwise. Then, a temporary metric $m_Hits@K$ is defined to compute the proportion of:

$$m_Hits@K = \frac{C}{N}. \quad (21)$$

The final metric is computed as the average of all the M temporary metrics:

$$M-Hits@K = \frac{1}{M} \cdot \sum_{m=1}^M m_Hits@K. \quad (22)$$

Obviously, our newly defined metric $M-Hits@K$ is more challenging than the traditional metric $Hits@K$ since the condition for counting a correct entity is much stricter.

A.4 Details of Baselines and MultiEA Variants

Regarding baselines, we select six representative supervised EA baselines that only leverage the structural information of candidate KGs.

- (1) **MTransE** [5] is a Trans-based EA method that uses TransE [3] to encode entities of each KG in a separate embedding space;
- (2) **IPTransE** [54] further uses PTransE [13] to encode KGs and propose an iterative alignment strategy;
- (3) **GCN-Align** [40] is the first GNN-based EA method that uses GCN [10] to encode two KGs;
- (4) **NAEA** [55] further uses GAT [35] to encode the structural information of KGs. Therefore, it learns entity embeddings by aggregating neighbor entities with different importance;
- (5) **KECG** [11] also uses GAT [35] to encode KGs, and additionally restricts the projection matrix to be a diagonal matrix, reducing the number of parameters and computations;
- (6) **PSR** [16] encodes KGs based on an orthogonal projection matrix and attention aggregation like ours. Differently, (1) it does not add self-connections, and (2) it directly concatenates all hops of neighborhood information.

Regarding variants, we use *+mean*, *+anchor*, and *+each* to denote three alignment strategies, respectively, and use *+infer* or *-infer* to indicate whether the module is equipped. By considering all the permutations, we set six variants for our MultiEA, as follows.

- (1) **MultiEA+mean-infer**;
- (2) **MultiEA+anchor-infer**;
- (3) **MultiEA+each-infer**;
- (4) **MultiEA+mean+infer**;
- (5) **MultiEA+anchor+infer**;
- (6) **MultiEA+each+infer**.

A.5 Implementation Details

For the constructed benchmark datasets of DBP-4 and DWY-3, we follow the convention to randomly select 30% elements of the seed

alignment set S to form the training set, and the rest are treated as the test set. We implement our MultiEA based on a PyTorch-based open-source EA toolkit, i.e., EAKit [48]. For baselines, we use their implementations provided in the EAKit package. All the experiments are conducted on our constructed benchmark datasets, and all the methods use the same dataset partitioning.

For our MultiEA, we use the same hyperparameter settings for both datasets. This is a challenging configuration since it can better reflect the sensitivity of hyperparameters w.r.t. datasets. All the model parameters are randomly initialized by the Xavier uniform distribution [7]. We adopt the Adam optimizer, and the learning rate is set to 0.01. The embedding dimensionalities of both entities and relations are set to 256. The non-linear activation function σ is set to the ELU function [2]. The margin hyperparameter λ is set to 1. The number of negative example groups, i.e., the hyperparameter η , is set to 10. We adopt early stopping for model training, and the number of patience steps is set to 10. For anchor-based variants, we treat DBP-4: en and DWY-3: dbp as the anchor KGs on the two datasets, respectively.

All the experiments are conducted on an NVIDIA TITAN RTX GPU with 24GB GPU memory and 128GB main memory.

A.6 Hyperparameter Study

In this subsection, we study the impact of MultiEA's four key hyperparameters on multiple KG alignment accuracy.

First, we study the balance hyperparameter γ introduced in the inference enhancement module, as described by Eqs. (16, 17). In practice, we only search the weight for the first term, and the weights for the other terms are set to the same value, thus compressing the hyperparameter search space. For one example of DBP-4 dataset, we can rewrite Eq. (17) as:

$$\begin{aligned} \tilde{S}^{en-zh} &= \gamma \cdot S^{en-zh} + \frac{1-\gamma}{2} \cdot S^{en-fr} \cdot S^{fr-zh} \\ &\quad + \frac{1-\gamma}{2} \cdot S^{en-ja} \cdot S^{ja-zh}. \end{aligned}$$

For another example of DWY-3 dataset, we can rewrite Eq. (16) as:

$$\tilde{S}^{dbp-wiki} = \gamma \cdot S^{dbp-wiki} + (1-\gamma) \cdot S^{dbp-yago} \cdot S^{yago-wiki}.$$

The sensitivity of γ on the two datasets is shown in Fig. 6(a) and Fig. 6(e), respectively. We can see that on both datasets, our MultiEA achieves the best performance when $\gamma = 0.2$. It means that the importance of the first-order similarity is 0.2 and the importance of the other higher-order similarities is 0.8. This indicates that incorporating high-order similarities is very beneficial for the multiple KG alignment task, demonstrating the effectiveness of our proposed inference enhancement module. This finding is also consistent with the observation in Section 5.1.

Second, we study the sensitivity of the margin hyperparameter λ introduced in the loss function as described by Eq. (9). The results are shown in Fig. 6(b) and Fig. 6(f). As we can see, the performance is very poor when $\lambda = 0$, because the model cannot separate the positive examples and the negative examples, causing the under-fitting issue. There is a clear inflection point when $\lambda = 1$, and thus in the other experiments, we set it to 1 by default. After that, the performance gradually declines, probably because the model suffers from the overfitting issue when the margin is too large.

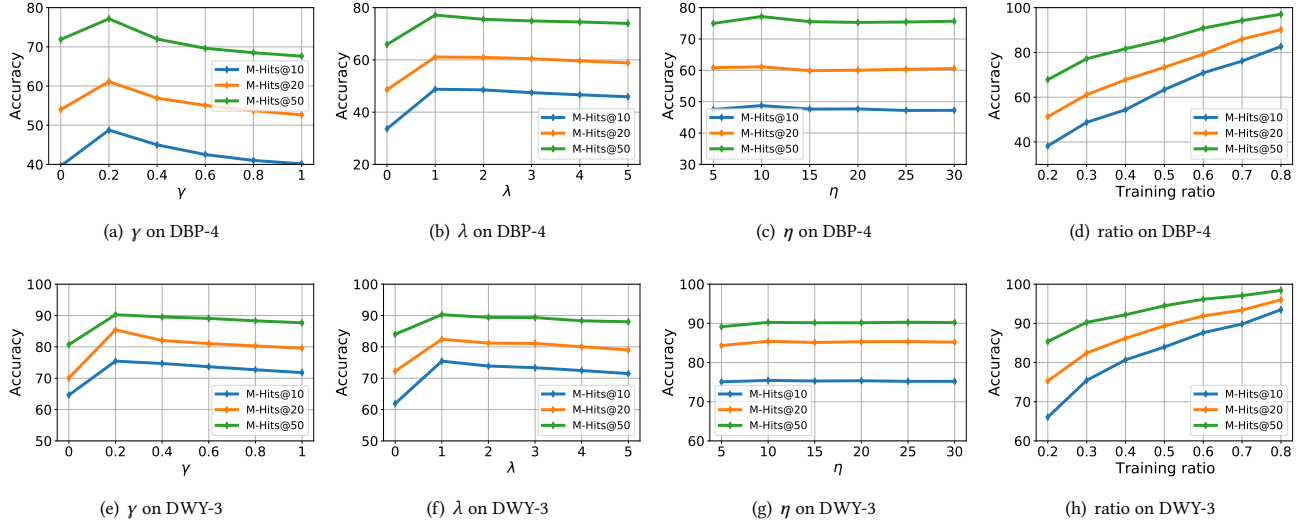


Figure 6: The sensitivity analysis of key hyperparameters.

Table 6: The *Hits@10* scores of pair-wise KG alignment.

Methods	DBP-4			DWY-3	
	en-zh	en-fr	en-ja	dbp-wiki	dbp-yago
MTransE	45.52%	50.13%	47.93%	60.54%	67.09%
MultiEA (MTransE)	45.93%	52.26%	50.19%	60.96%	67.36%
Relative Gain $\uparrow$	+0.90%	+4.25%	+4.72%	+0.69%	+0.40%
IPTransE	56.23%	57.79%	60.66%	75.96%	79.32%
MultiEA (IPTransE)	58.52%	59.35%	62.43%	77.16%	82.17%
Relative Gain $\uparrow$	+4.07%	+2.70%	+2.92%	+1.58%	+3.59%
GCN-Align	62.03%	68.05%	63.96%	77.58%	86.00%
MultiEA (GCN-Align)	62.41%	70.37%	65.22%	78.56%	88.02%
Relative Gain $\uparrow$	+0.61%	+3.41%	+1.97%	+1.26%	+2.35%
KECG	67.19%	71.29%	67.51%	80.40%	88.65%
MultiEA (KECG)	68.03%	73.10%	69.38%	80.73%	89.78%
Relative Gain $\uparrow$	+1.25%	+2.54%	+2.77%	+0.41%	+1.27%
NAEA	66.41	71.43	67.83	80.67	88.76
MultiEA (NAEA)	68.18%	74.17%	70.95%	81.35%	89.37%
Relative Gain $\uparrow$	+2.67%	+3.84%	+4.60%	+0.84%	+0.69%
PSR	67.83%	73.04%	69.93%	82.41%	90.26%
MultiEA (PSR)	70.04%	75.93%	71.81%	83.97%	90.79%
Relative Gain $\uparrow$	+3.25%	+3.96%	+2.69%	+1.90%	+0.59%

Third, we investigate the sensitivity of η , which has been introduced to describe the groups of negative examples. As shown in Fig. 6(c) and Fig. 6(g), our model is not sensitive to this hyperparameter. Therefore, in practice, we set η to 10 to save computational resources.

Finally, we show the model performance w.r.t. the training ratio in Fig. 6(d) and Fig. 6(h). As expected, the training ratio shows a clear positive effect on improving the model performance, which is

consistent with the findings of many previous studies, e.g., [11, 16, 25, 27, 41, 42].

A.7 More Pair-wise KG Alignment Results.

Table 6 shows the *Hits@10* scores of pair-wise KG alignment, which serves as a supplement to the experimental results in Section 5.4, and from these results, we can draw the same conclusion as in Section 5.4.